

To-Do List in Terminal

Aanjneya Singh

25BAI10885

PROBLEM STATEMENT-

The lack of tracking all the daily tasks throughout the day and keeping up with the schedule.

Solution-

A simple Java command-line programme which can keep track of the user's tasks according to the time, record whether each task has been completed, and store the information for later use.

The programme should be able to add, remove, update, search, display the data and create a new file to store task records.

1. Project Overview

This project is a command-line based To-Do List / Task Manager application written in Java. The main purpose is to allow users to create, view, search, update, and delete personal task records efficiently. The application uses Java's object serialization mechanism to store TaskRecord objects in a binary file named todo.dat. A menu-driven interface allows the user to repeatedly select the required operation from the terminal.

FUNCTIONS

Create a New Data File: The system allows the user to create a new set of task records and save them to the todo.dat file.

Add Task: The system allows the user to append a new task. Serial numbers are assigned automatically, while time, date, year, completion status and task description are stored.

Display Tasks: The system reads all TaskRecord objects from the file and displays them in a formatted table.

Search Tasks: The user can search for a task by serial number or find all tasks scheduled on a specified day, month and year.

Update Task: The user selects a task using its serial number and can update its time, date, year, completion status and description.

Delete Task: The user can select a task by serial number and remove that record from the stored data.

Mark Task as Completed: The completion status of a task can be changed through the update operation.

Menu Navigation: A clear interactive terminal menu provides access to all major functions.

Input Validation: The program checks numeric ranges, empty task descriptions and excessive task length before storing data.

Error Reporting: Invalid numeric input, missing records and file-writing problems are reported to the user.

NON FUNCTIONAL REQUIREMENTS-

- Usability: The terminal menu provides simple text-based navigation.
- Reliability: Records are read from and written to the data file using Java object streams.
- Maintainability: The program is organized into methods for separate operations such as adding, searching, updating and deleting.
- Data Persistence: Task information remains available after the program is closed because it is stored in todo.dat.
- Input Handling: Scanner is used for interactive terminal input and invalid numeric input is handled using exceptions.

- Extensibility: The TaskRecord class and separate methods make it possible to add more task-related features later.

2. TECHNOLOGIES / LIBRARIES USED

Java: The primary programming language used to implement the application.

java.io.*: Provides File, FileInputStream, FileOutputStream, ObjectInputStream, ObjectOutputStream, IOException and Serializable for file handling and object serialization.

java.util.*: Provides Scanner, List, ArrayList and other utility classes used for input and collection management.

Scanner: Reads user input from the terminal.

ArrayList: Stores TaskRecord objects dynamically while records are being processed.

Serializable: Allows TaskRecord objects to be converted into a byte stream for storage in todo.dat.

ObjectInputStream / ObjectOutputStream: Read and write serialized TaskRecord objects.

3. DATA STRUCTURE

Each task is represented by an object of the inner TaskRecord class. The class implements Serializable so that objects can be stored directly in the binary file.

- Serial Number (sno) – int
- Hour (time) – int
- Day – int
- Month – int
- Year – int
- Completion Status (completed) – boolean
- Task Description (task) – String

The records are maintained in a List<TaskRecord>, implemented using ArrayList. The objects are written individually to the file todo.dat using ObjectOutputStream and read back using ObjectInputStream.

4. KEY FEATURES AND FUNCTIONALITY

Create File: `createfile()` creates a new collection of records, validates the entered values and writes the resulting `TaskRecord` objects to `todo.dat`.

Add Task: `adding()` reads the existing records, assigns the next serial number, validates the input and appends the new record.

Display All Tasks: `displaying()` reads all records and sends them to `printFancyTable()`, which produces a formatted terminal table.

Search Task: `searching()` supports two search modes: serial number and date. Matching records are displayed in the same table format.

Update Task: `updating()` locates a record by serial number, accepts replacement values and preserves existing values when optional fields are left blank.

Delete Task: `deleting()` removes the selected serial number from the list and rewrites the remaining records to `todo.dat`.

Interactive Menu: `menu()` repeatedly displays available commands: new, read, search, update, delete, add and quit.

Input and Error Handling: `NumberFormatException` is handled for invalid numeric entries. Range checks and empty-input checks provide feedback before records are saved.

Terminal Table Output: `printFancyTable()` calculates column widths and uses box-drawing characters to present task records in a readable table.

5. OBJECT-ORIENTED PROGRAMMING CONCEPTS USED

Class and Object: `TaskRecord` is a class used to create individual task objects.

Encapsulation: `TaskRecord` fields and the methods that operate on records are organized within classes/methods.

Abstraction: Helper methods such as `readRecords()`, `writeRecords()`, `input()` and `sleep()` hide repeated implementation details from the main menu.

Collections: `ArrayList` provides dynamic storage for multiple `TaskRecord` objects.

Exception Handling: try-catch blocks handle invalid numeric input and file-related exceptions.

File handling flow:

todo.dat → ObjectInputStream → TaskRecord objects → ArrayList → processing
ArrayList → ObjectOutputStream → todo.dat

7. IMPORTANT JAVA METHODS

main(): Entry point of the program and starts the interactive menu.

menu(): Controls repeated user interaction and calls the appropriate operation.

readRecords(): Reads serialized TaskRecord objects from todo.dat into an ArrayList.

writeRecords(): Writes the current collection of TaskRecord objects back to todo.dat.

printFancyTable(): Displays records with calculated column widths and table borders.

createfile(), adding(), displaying(), searching(), updating(), deleting(): Perform the main task-management operations.

8. TESTING APPROACH

The program can be tested through repeated terminal executions. Each major menu option is tested independently and then the complete workflow is tested from creating a new file through adding, reading, searching, updating, deleting and quitting.

Create File: Enter valid records and verify that they are stored in todo.dat.

Display: Use read and verify that all stored records appear in a formatted table.

Search by Serial Number: Enter an existing and a non-existing serial number and check the displayed result/message.

Search by Date: Enter a date containing tasks and a date containing no tasks.

Update: Select an existing serial number, modify values and use read to verify the changes.

Delete: Delete an existing record and verify that it no longer appears.

Add: Append a new task and verify its automatically assigned serial number.

Invalid Input: Enter invalid numbers, out-of-range dates, empty task descriptions and excessively long task descriptions.

Quit: Select quit and verify that the program terminates.

9. SAMPLE TERMINAL OUTPUT

```
Enter (new) to create a new file
Enter (read) to display all the tasks or records
Enter (search) to search for a record using serial number or date
Enter (update) to update a record
enter (delete) to delete a record
Enter (add) to append a record at the end
Enter (quit) to exit the programme
```

```
-----
PLEASE ENTER YOUR CHOICE OF FUNCTION TO PROCEED:new
enter number of records you wish to enter: 2
enter the hour you want to do the task in(0-24)11
enter the date of the task: 11
enter the month of task completing: 11
enter year: 2025
enter task you want to achieve: eat
enter the hour you want to do the task in(0-24)13
enter the date of the task: 11
enter the month of task completing: 11
enter year: 2025
enter task you want to achieve: sleep
```

10. POSSIBLE ENHANCEMENTS

- Improve date validation so that impossible calendar dates, such as 31 February, are rejected.
- Use `LocalDate` and `LocalTime` from `java.time` for stronger date and time handling.
- Replace individual serialized objects with a single serialized `List<TaskRecord>` for simpler bulk updates and deletions.
- Add task priorities and categories.
- Add due-date notifications or reminders.
- Add sorting by date, time, priority or completion status.
- Add a graphical user interface using `JavaFX` or `Swing`.
- Port the application to a web-based interface with a database backend.

11. CONCLUSION

The Java To-Do List in Terminal is a command-line task management application that demonstrates practical Java programming concepts. It combines classes and objects, `ArrayList` collections, file handling, object serialization, exception handling, input validation and menu-driven interaction. The application provides the basic operations required for maintaining personal tasks and creates a foundation that can be extended with improved date handling, priorities, notifications, graphical interfaces or database storage.

12. SOURCE PROGRAM

The project is implemented in the Java source file `ToDoList.java`. The central data model is the serializable `TaskRecord` class, while the program's operations are organized into methods for file creation, display, search, update, deletion, addition and menu navigation.

PROJECT OUTPUT

Java To-Do List Terminal Application

File used for persistent task storage: `todo.dat`